Add Whack A Mole Features

Generated: 2026-11-12T17:28:00.365Z • Source: Shared ChatGPT link

USER

https://github.com/NayanKaushik25/48_whack-a-mole

These are the tasks containing features I need to add

ASSISTANT

I've reviewed the current code in your . It's a small Pygame project with a GameEngine class managing the game state and a Hole class managing each mole.

We'll make the changes incrementally, keeping them small enough for you to understand, test, and critically review—the assignment specifically asks for an iterative LLM-assisted process.

Implementation plan

1. Fix collision detection

Make each click register at most one successful hit. Use the actual mole area rather than oversized, overlapping rectangular hitboxes.

---

2. Add a game-over screen

Display the final score and wait for the player's next choice instead of printing the result in the terminal.

---

3. Add replay and difficulty

Offer Easy, Medium, Hard, and Quit, with difficulty controlling mole spawn frequency and how long moles stay visible.

---

4. Add sound effects

Play distinct feedback for a successful hit, a miss, and the end of the round.

Let's start with Task 1

There are two problems in the current collision logic:

- Hole.hit_size is 150, so each hole has a large rectangular hitbox even though the visible mole has a radius of only 32 pixels.
- _handle_click() iterates through **every hole** without stopping after a successful hit. If overlapping hitboxes contain the click, multiple moles can be whacked.

Your first change

In game/hole.py, add this method inside the Hole class:

```
def contains_point(self, pos, radius=32):
    x, y = pos
    dx = x - self.center_x
    dy = y - self.center_y

    return dx * dx + dy * dy <= radius * radius
```

This checks whether the click is inside the circular mole area using squared distance, avoiding an unnecessary square root.

Next, replace _handle_click() in game/game_engine.py with:

```
def _handle_click(self, pos):
    for hole in self.holes:
        if hole.active and hole.contains_point(pos):
            if hole.whack():
                self.score += 1
                return

    self.misses += 1
```

Review the change

NOTICE

three important details:

1. Only **active moles** can be hit.
2. A successful click awards exactly one point because the method returns immediately.
3. A click that hits no active mole increments misses exactly once.

One subtle point: this uses a radius of 32, matching the currently drawn mole. The dark-brown hole has a radius of 40, but clicking its empty border should not count as whacking a mole.

Make these two changes, run python main.py, and test them. Once you've checked that the collision bug is fixed, tell me done and we'll implement Task 2 without jumping ahead.

USER

ok It is working properly, It only awards one point per mole no matter how fast I click. Lets go to task 2

ASSISTANT

Nice! That's exactly what we wanted for Task 1. The important fix is that a successful click now whacks one active mole and immediately returns, so rapid clicking cannot award multiple points from a single event.

Task 2: Implement the Game Over screen

Currently, when the timer reaches zero, GameEngine.update() sets self.game_over = True. The render() method then prints the final score to the terminal once, but the game window doesn't display a proper end screen.

We'll fix this in **game/game_engine.py only** for now.

Step 1: Add a second font

Inside __init__(), directly below your existing font initialization:

```
self.font = pygame.font.SysFont("Arial", 28)
```

add:

```
self.title_font = pygame.font.SysFont("Arial", 42, bold=True)
```

This gives us a larger font for the game-over heading.

Step 2: Replace the game-over logic in render()

At the bottom of render(), remove this existing block:

```
if self.game_over and not getattr(self, "_game_over_logged", False):
    print("Time's up! Final score:", self.score)
    self._game_over_logged = True
```

Replace it with:

```
if self.game_over:
    overlay = pygame.Surface((self.width, self.height))
    overlay.set_alpha(210)
    overlay.fill((30, 30, 30))
    screen.blit(overlay, (0, 0))

    title = self.title_font.render(
        "GAME OVER", True, (255, 255, 255)
    )
    final_score = self.font.render(
        f"Final Score: {self.score}", True, (255, 255, 255)
    )
    instruction = self.font.render(
        "Press any key to continue", True, (200, 200, 200)
    )

    screen.blit(
        title,
        title.get_rect(center=(self.width // 2, self.height // 2 - 70))
    )
    screen.blit(
        final_score,
        final_score.get_rect(center=(self.width // 2, self.height // 2))
    )
    screen.blit(
        instruction,
        instruction.get_rect(center=(self.width // 2, self.height // 2 + 60))
    )
```

This draws a dark translucent overlay over the existing game, followed by the heading, final score, and an instruction.

Step 3: Make the game wait for input

We need to ensure the game doesn't keep processing mouse clicks after the timer expires, and that the player can acknowledge the end screen.

In handle_event(), replace the current method with:

```
def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN or event.type == pygame.MOUSEBUTTONDOWN:
            self.waiting_for_input = True
        return

    if event.type == pygame.MOUSEBUTTONDOWN:
        self._handle_click(event.pos)
```

Then, inside __init__(), add:

```
self.waiting_for_input = False
```

For now, this records that the player has acknowledged the screen. In Task 3, we'll connect that input to the replay/difficulty menu.

One detail to understand: the return in handle_event() is important. It prevents clicks on the game-over screen from changing the final score.

Step 4: Test it

Run:

```
python main.py
```

Wait for the 30-second timer to expire. Verify that:

- The game-over overlay appears.
- The final score is displayed correctly.
- The terminal no longer prints the final score.
- Clicking or pressing a key is accepted without changing the final score.

Once this works, tell me done. Then we'll move to Task 3 and replace the temporary input acknowledgement with a proper **Easy / Medium / Hard / Quit** menu and replay functionality.

USER

yea the press any key to continue feature doesn't work

ASSISTANT

The issue is that we set self.waiting_for_input = True when a key is pressed, but **nothing actually uses that variable yet**. So the input is registered, but the game doesn't respond to it.

Since Task 3 will add the replay menu, let's fix this cleanly without implementing replay prematurely.

Step 1: Update handle_event() in game/game_engine.py

Replace your current method with:

```
def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN or event.type == pygame.MOUSEBUTTONDOWN:
            self.waiting_for_input = True
        return

    if event.type == pygame.MOUSEBUTTONDOWN:
        self._handle_click(event.pos)
```

If this is already what you have, that's fine—we'll keep it.

Step 2: Display a response when the player presses a key

Inside render(), in the existing if self.game_over: block, replace the instruction-rendering section with:

```
if self.waiting_for_input:
    instruction = self.font.render(
        "Input received!", True, (100, 255, 100)
    )
else:
    instruction = self.font.render(
        "Press any key to continue", True, (200, 200, 200)
    )
```

Keep the existing screen.blit() call for instruction directly below it.

Step 3: Test it

Run the game again and wait for the timer to expire. Press any key.

The instruction should change to **"Input received!"**. This confirms that Pygame receives the input and that the game-over screen responds to it.

Important: This is only a temporary acknowledgement. The screen won't disappear yet. In Task 3, we'll make the input open the difficulty-selection menu, which is the meaningful next action after game over.

USER

just give me the complete render function

ASSISTANT

Replace your entire render() function in game/game_engine.py with this:

```
def render(self, screen):
    for hole in self.holes:
        pygame.draw.circle(
            screen, DARK_BROWN,
            (hole.center_x, hole.center_y), 40
        )

        if hole.active:
            pygame.draw.circle(
                screen, MOLE_BROWN,
                (hole.center_x, hole.center_y), 32
            )

    score_text = self.font.render(
        f"Score: {self.score}", True, BLACK
    )
    screen.blit(score_text, (10, 10))

    seconds_left = max(0, self.time_left_frames // 60)
    timer_text = self.font.render(
        f"Time: {seconds_left}s", True, BLACK
    )
    screen.blit(timer_text, (self.width - 140, 10))

    if self.game_over:
        overlay = pygame.Surface((self.width, self.height))
        overlay.set_alpha(210)
        overlay.fill((30, 30, 30))
        screen.blit(overlay, (0, 0))

        title = self.title_font.render(
            "GAME OVER", True, (255, 255, 255)
        )
        final_score = self.font.render(
            f"Final Score: {self.score}", True, (255, 255, 255)
        )

        if self.waiting_for_input:
            instruction = self.font.render(
                "Input received!", True, (100, 255, 100)
            )
        else:
            instruction = self.font.render(
                "Press any key to continue",
                True, (200, 200, 200)
            )

        screen.blit(
            title,
            title.get_rect(
```

```
            center=(self.width // 2, self.height // 2 - 70)
        )
    )
    screen.blit(
        final_score,
        final_score.get_rect(
            center=(self.width // 2, self.height // 2)
        )
    )
    screen.blit(
        instruction,
        instruction.get_rect(
            center=(self.width // 2, self.height // 2 + 60)
        )
    )
```

Make sure you have these two initializations in __init__():

```
self.title_font = pygame.font.SysFont("Arial", 42, bold=True)
self.waiting_for_input = False
```

This version displays the final score and changes the instruction to Input received! when a key or mouse button is pressed. The screen stays visible; we'll implement the actual continuation through the difficulty menu in Task 3.

USER

Ok Task 3

ASSISTANT

Log in to view this content.Let's implement **Task 3: Replay and difficulty selection**.

We'll turn the game-over screen into a menu with four options: **Easy, Medium, Hard, and Quit**. Selecting a difficulty starts a fresh 30-second round with a reset score and new mole spawn/visibility settings. We'll keep the changes in your existing Pygame structure rather than introducing another module.

The key design change is to use a game state: "playing", "game_over", or "menu". This makes the transition from the finished round to replay much cleaner than using waiting_for_input.

Step 1: Replace game/game_engine.py

To keep the state transitions consistent, replace the entire file with the following version. It includes your Task 1 collision fix and Task 2 game-over screen as well.

```
import pygame
import random
from .hole import Hole

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
GRAY = (200, 200, 200)
GREEN = (100, 220, 100)

class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 80) // (rows + 1)

        for r in range(rows):
            for c in range(cols):
                cx = spacing_x * (c + 1)
                cy = 80 + spacing_y * (r + 1)
                self.holes.append(Hole(cx, cy))

        # Difficulty settings: spawn chance and mole visibility
        self.difficulties = {
            "Easy": (0.01, 60),
            "Medium": (0.02, 45),
            "Hard": (0.04, 30),
        }

        self.difficulty = "Medium"
        self.spawn_chance, self.mole_up_frames = (
            self.difficulties[self.difficulty]
        )

        self.round_seconds = 30
        self.time_left_frames = self.round_seconds * 60

        self.score = 0
        self.misses = 0

        self.font = pygame.font.SysFont("Arial", 28)
        self.title_font = pygame.font.SysFont(
            "Arial", 42, bold=True
        )
        self.small_font = pygame.font.SysFont("Arial", 20)
```

```
        # States: playing, game_over, menu
        self.state = "playing"
        self.game_over = False
        self.quit_requested = False

    def start_new_round(self, difficulty):
        self.difficulty = difficulty
        self.spawn_chance, self.mole_up_frames = (
            self.difficulties[difficulty]
        )

        self.score = 0
        self.misses = 0
        self.time_left_frames = self.round_seconds * 60

        for hole in self.holes:
            hole.active = False
            hole.timer = 0

        self.game_over = False
        self.state = "playing"

    def handle_event(self, event):
        if event.type == pygame.QUIT:
            self.quit_requested = True
            return

        if self.state == "playing":
            if event.type == pygame.MOUSEBUTTONDOWN:
                self._handle_click(event.pos)

        elif self.state == "game_over":
            if (
                event.type == pygame.KEYDOWN
                or event.type == pygame.MOUSEBUTTONDOWN
            ):
                self.state = "menu"

        elif self.state == "menu":
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_1:
                    self.start_new_round("Easy")
                elif event.key == pygame.K_2:
                    self.start_new_round("Medium")
                elif event.key == pygame.K_3:
                    self.start_new_round("Hard")
                elif event.key in (pygame.K_4, pygame.K_ESCAPE):
                    self.quit_requested = True

            elif event.type == pygame.MOUSEBUTTONDOWN:
                x, y = event.pos
```

```
            # Match the button positions in render()
            for i, difficulty in enumerate(
                ["Easy", "Medium", "Hard"]
            ):
                button = pygame.Rect(
                    self.width // 2 - 120,
                    220 + i * 65,
                    240,
                    48,
                )

                if button.collidepoint(x, y):
                    self.start_new_round(difficulty)
                    return

            quit_button = pygame.Rect(
                self.width // 2 - 120,
                415,
                240,
                48,
            )

            if quit_button.collidepoint(x, y):
                self.quit_requested = True

    def _handle_click(self, pos):
        for hole in self.holes:
            if hole.active and hole.contains_point(pos):
                if hole.whack():
                    self.score += 1
                    return

        self.misses += 1

    def handle_input(self):
        pass

    def update(self):
        if self.state != "playing":
            return

        self.time_left_frames -= 1

        if self.time_left_frames <= 0:
            self.time_left_frames = 0
            self.game_over = True
            self.state = "game_over"

            for hole in self.holes:
                hole.active = False
                hole.timer = 0
```

```
            return

        for hole in self.holes:
            hole.update()

            if (
                not hole.active
                and random.random() < self.spawn_chance
            ):
                hole.pop_up(self.mole_up_frames)

    def render(self, screen):
        # Draw the game board
        for hole in self.holes:
            pygame.draw.circle(
                screen,
                DARK_BROWN,
                (hole.center_x, hole.center_y),
                40,
            )

            if hole.active:
                pygame.draw.circle(
                    screen,
                    MOLE_BROWN,
                    (hole.center_x, hole.center_y),
                    32,
                )

        score_text = self.font.render(
            f"Score: {self.score}", True, BLACK
        )
        screen.blit(score_text, (10, 10))

        seconds_left = max(0, self.time_left_frames // 60)
        timer_text = self.font.render(
            f"Time: {seconds_left}s", True, BLACK
        )
        screen.blit(
            timer_text,
            (self.width - 140, 10),
        )

        if self.state == "game_over":
            self._draw_overlay(screen)

            title = self.title_font.render(
                "GAME OVER", True, WHITE
            )
            final_score = self.font.render(
                f"Final Score: {self.score}", True, WHITE
            )
```

```
            instruction = self.small_font.render(
                "Press any key or click to continue",
                True,
                GRAY,
            )

            screen.blit(
                title,
                title.get_rect(
                    center=(
                        self.width // 2,
                        self.height // 2 - 70,
                    )
                ),
            )
            screen.blit(
                final_score,
                final_score.get_rect(
                    center=(
                        self.width // 2,
                        self.height // 2,
                    )
                ),
            )
            screen.blit(
                instruction,
                instruction.get_rect(
                    center=(
                        self.width // 2,
                        self.height // 2 + 60,
                    )
                ),
            )

        elif self.state == "menu":
            self._draw_overlay(screen)

            title = self.title_font.render(
                "PLAY AGAIN?", True, WHITE
            )
            screen.blit(
                title,
                title.get_rect(
                    center=(self.width // 2, 145)
                ),
            )

            for i, difficulty in enumerate(
                ["Easy", "Medium", "Hard"]
            ):
                button = pygame.Rect(
                    self.width // 2 - 120,
```

```
                220 + i * 65,
                240,
                48,
            )

            pygame.draw.rect(
                screen, DARK_BROWN, button, border_radius=8
            )

            label = self.font.render(
                difficulty, True, WHITE
            )
            screen.blit(
                label,
                label.get_rect(center=button.center),
            )

        quit_button = pygame.Rect(
            self.width // 2 - 120,
            415,
            240,
            48,
        )
        pygame.draw.rect(
            screen, (120, 45, 45),
            quit_button, border_radius=8
        )

        quit_label = self.font.render(
            "Quit", True, WHITE
        )
        screen.blit(
            quit_label,
            quit_label.get_rect(center=quit_button.center),
        )

        hint = self.small_font.render(
            "Press 1, 2, 3 to play | 4 to quit",
            True,
            GRAY,
        )
        screen.blit(
            hint,
            hint.get_rect(
                center=(self.width // 2, 490)
            ),
        )

    def _draw_overlay(self, screen):
        overlay = pygame.Surface(
            (self.width, self.height)
        )
```

```
        overlay.set_alpha(220)
        overlay.fill((30, 30, 30))
        screen.blit(overlay, (0, 0))
```

Step 2: Update main.py

We need the main loop to stop when the player chooses Quit from the menu.

Find this part:

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    engine.handle_event(event)
```

Replace it with:

```
for event in pygame.event.get():
    engine.handle_event(event)

    if engine.quit_requested:
        running = False
```

This lets the engine handle both the window-close event and the menu's Quit option.

Step 3: Run and test

Run the game:

```
python main.py
```

Test this sequence:

1. Play until the 30-second timer expires.
2. Verify that the final score appears.
3. Press a key or click to open the replay menu.
4. Select Easy, Medium, or Hard by clicking the button, or press 1, 2, or 3.
5. Verify that the score resets to zero, the timer returns to 30 seconds, and the new difficulty affects mole spawning and visibility.
6. Return to the menu after another round and choose Quit, or press 4.

One implementation detail worth noticing: the difficulty settings change two independent parameters. Easy uses a lower spawn probability and longer visibility, while Hard uses a higher spawn probability and shorter visibility. This gives each mode a distinct feel without changing the scoring rules.

Once this works, Task 4 will add sound feedback for successful hits, missed clicks, and the round ending.

USER

ok so what is the default difficulty in the beginning?

ASSISTANT

The default difficulty is **Medium**.

In game/game_engine.py, these lines set it:

```
self.difficulty = "Medium"
self.spawn_chance, self.mole_up_frames = (
    self.difficulties[self.difficulty]
)
```

So the initial game starts with these settings:

- **Spawn chance:** 0.02 (2% per inactive hole, per frame)
- **Mole visibility:** 45 frames (0.75 seconds at 60 FPS)
- **Round duration:** 30 seconds

The player starts playing immediately on Medium. The difficulty-selection menu appears only after the first round ends and the player continues.

USER

ok

ASSISTANT

Great! Task 3 is covered. Let's move on to **Task 4: Sound Feedback** when you're ready.

We'll add three sound effects:

- Successful whack
- Missed click
- Round ending

We'll use Pygame's built-in pygame.mixer and keep the implementation simple.

USER

Alright Task 4

ASSISTANT

Log in to view this content.Let's finish **Task 4: Sound Feedback**! We'll add three distinct sounds to your Whack-a-Mole game:

- **Successful whack:** a short, high-pitched pop.
- **Missed click:** a lower-pitched sound.
- **Round ending:** a short descending tone.

We'll generate the sounds directly using Pygame, so you won't need to download audio files or install another dependency. We'll also initialize the mixer safely so the game can still run if audio isn't available on your system.

Step 1: Add sound generation to game/game_engine.py

At the top of the file, add these imports:

```
import math
from array import array
```

Inside your GameEngine class, add the following methods. They generate short audio tones in memory, so you don't need separate .wav files.

```
def _make_tone(self, start_freq, duration, volume=0.3, end_freq=None):
    if not pygame.mixer.get_init():
        return None

    sample_rate, _, _ = pygame.mixer.get_init()
    samples = array("h")
    total_samples = int(sample_rate * duration)

    for i in range(total_samples):
        progress = i / max(1, total_samples - 1)

        if end_freq is None:
            frequency = start_freq
        else:
            frequency = start_freq + (
                end_freq - start_freq
            ) * progress

        # Fade out to avoid an abrupt audio click.
        envelope = min(1.0, (1.0 - progress) * 10)
        value = int(
            32767
            * volume
            * envelope
            * math.sin(
                2 * math.pi * frequency * i / sample_rate
            )
        )
        samples.append(value)

    return pygame.mixer.Sound(buffer=samples.tobytes())

def _init_sounds(self):
    self.hit_sound = None
    self.miss_sound = None
    self.game_over_sound = None

    try:
        if not pygame.mixer.get_init():
            pygame.mixer.init()

        self.hit_sound = self._make_tone(1000, 0.08, 0.25)
        self.miss_sound = self._make_tone(180, 0.12, 0.2)
        self.game_over_sound = self._make_tone(
            700, 0.5, 0.25, end_freq=250
        )
    except pygame.error:
        # Allow the game to run without audio hardware.
        self.hit_sound = None
```

```
        self.miss_sound = None
        self.game_over_sound = None
```

Step 2: Initialize the sounds

Inside __init__(), after your font initialization, add:

```
self._init_sounds()
```

This creates the sounds once when the game starts, rather than generating them every time a player clicks.

Step 3: Play the hit and miss sounds

In _handle_click(), update the successful-hit and miss logic to:

```
def _handle_click(self, pos):
    for hole in self.holes:
        if hole.active and hole.contains_point(pos):
            if hole.whack():
                self.score += 1

                if self.hit_sound:
                    self.hit_sound.play()

                return

    self.misses += 1

    if self.miss_sound:
        self.miss_sound.play()
```

The hit sound plays only after a successful whack. Clicking empty space or an inactive hole plays the miss sound.

Step 4: Play the round-ending sound

In update(), find the code that sets the game-over state when the timer reaches zero. In your Task 3 implementation, it should look like this:

```
if self.time_left_frames <= 0:
    self.time_left_frames = 0
    self.game_over = True
    self.state = "game_over"

    for hole in self.holes:
        hole.active = False
        hole.timer = 0

    if self.game_over_sound:
        self.game_over_sound.play()

    return
```

This plays the ending sound only when the round transitions into the game-over state.

Step 5: Test everything

Run:

```
python main.py
```

Check these three cases:

Action	Expected sound
Click an active mole	Short, high-pitched pop
Click empty space	Low-pitched tone
Timer reaches zero	Descending tone

If you don't hear anything, check your system volume and whether Pygame's audio mixer initializes successfully.

That's all four tasks implemented in principle! Once the sound effects work, you can record your required 10-second before-and-after gameplay clips and submit your ChatGPT conversation link as described in the README.

USER

Alright everything works Thanks

ASSISTANT

You're welcome! Great job getting all four tasks working.

You've now got a solid little Pygame project with collision detection, a game-over screen, difficulty selection, replay functionality, and sound effects.

Before submitting, just make sure you:

- [ ] Record the **10-second before video** showing the original collision bug.
- [ ] Record the **10-second after video** demonstrating the fixed collision detection and new features.
- [ ] Include the link to this ChatGPT conversation with your complete chat history.
- [ ] Commit and push your final changes to GitHub.

One last tip: keep your Git commits organized by task if you haven't already. It makes your iterative development process much easier to demonstrate.

Nice work sticking with it through all four tasks!

If you want, I can:

- Add sound volume control menu option
- Implement score multiplier based on difficulty
- Add visual feedback for hits and misses